

openSUSE
Asia Summit 2026
Yogyakarta, October 3 - 4

Linux Kernel External Scheduler (sched_ext)

Tonny Adhi Sabastian
t.a.sabastian@gmail.com

Yogyakarta | October, 2026

About Me

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Tonny Adhi Sabastian

System Engineer, System / Solution Architect, DevOps, Embedded Software Engineer, Half Academician

Interest : System Performance, Distributed System, Linux Kernel, eBPF, IoT


@ragnalinux


@tonnyadhi


@tonny-adhi-sabastian


@ragnalinux

Universitas Indonesia (2009 - 2018)

- **System Administrator - kambing.ui.ac.id**
- **Lecturer for Distributed System, Modern IT Infrastructure, Operating System, System Programming, F/OSS Development**
- **Research Engineer - Pervasive Computing Lab, Network Engineering**

Solution Architect & Senior DevOps - MAPAN (GOTO Group) - 2018 - 2021

Principal System Engineer - Stockbit - 2022 - 2023

Tech Lead Platform Engineering - Pintu - 2024

Embedded System Tech Lead - Netzme Kreasi Indonesia - 2025

VP Of Engineering - Embedded System, Infrastructure, Security - Netzme Kreasi Indonesia - 2026 - Now

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

01

What is a CPU Scheduler ? + eBPF 101

What is a CPU Scheduler and its task to resources ,management ? We will also take a peek into eBPF as one of key component for modern CPU Scheduler Programming

02

sched_ext

Lets dive for a while into sched_ext. The modern way to tackle CPU scheduling problem without the hassle of rebooting and also rapid development capabilities

03

The Three Scheduler

We will take a look into several "toy" scheduler that i played into for this talk

04

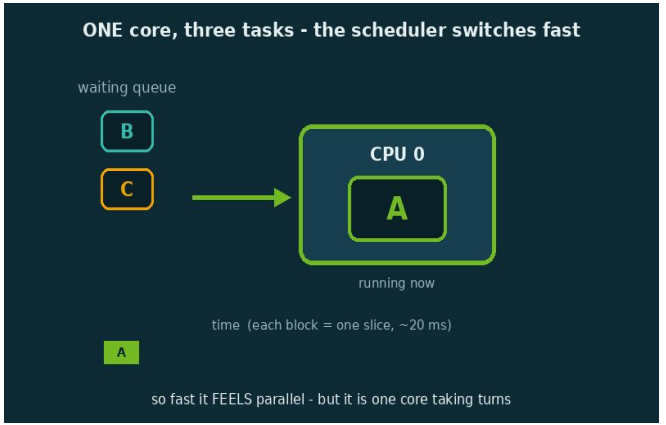
Demo : Lets Run The Scheduler

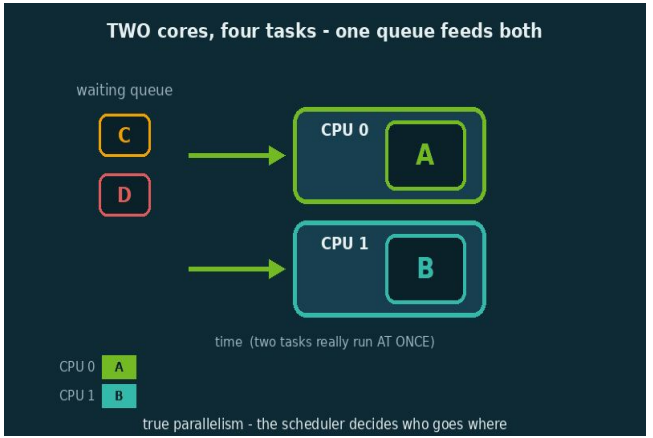
Lets run and observe our scheduler

CPU Scheduler

What is a Scheduler ?

- Kernel component that determines :
  - **Where** each task needs to run
  - **When** each task needs to run
  - **How long** each task needs to run - time slicing





Goals and Challenge

- Fairness : A task receives fair amount of share on CPU time
- Optimization of System Resources
- Low Overhead : Scheduler should run as fast as possible
- Generalization : Scheduler should work on all architectures for every workload
  - Earlier CPU architecture much more simpler
  - Currently we are facing more complex architecture (NUMA Nodes, Core Complex, Core Complex Dies, Performance Cores , Energy Efficient Cores, etc)

Scheduling in Linux

- One scheduler to rule them all
  - CFS - Completely Fair Schedule < 6.6
  - EEVDF - Earliest Eligible Virtual Deadline First >= 6.6
- Schedulers are built into Kernel - Really difficult to conduct experiments
  - It have to be compiled into the kernel
  - Might crashed your kernel if you don't fully safety when implementing changes
- Really difficult for streaming

CFS - Proportional Weight based CPU Allocation - Fairness

- Each task T_i has a weight of w_i
- Runtime assigned to task T_i is proportional to its weight w_i divided by the sum of all the runnable task weight

$$runtime(T_i) = \int_{t_0}^{t_1} \frac{w_i}{\sum_{j=0}^{N} w_j} dt \simeq \frac{w_i}{\sum_{j=0}^{N} w_j} \cdot (t_1 - t_0)$$

- Andrea Righi - NVIDIA -

CFS - Proportional Weight based CPU Allocation - Fairness

Example - 2 Tasks with weight of each task equal 1

50%

$$runtime(T_0) = \frac{w_0}{\sum_{j=0}^{N} w_j} \cdot (t_1 - t_0) = \frac{1}{1+1} \cdot (t_1 - t_0) = \frac{1}{2} \cdot (t_1 - t$$

50%

$$runtime(T_1) = \frac{w_1}{\sum_{j=0}^{N} w_j} \cdot (t_1 - t_0) = \frac{1}{1+1} \cdot (t_1 - t_0) = \frac{1}{2} \cdot (t_1 - t$$

EEVDF : Virtual Runtime

- Charge each tak a runtime proportional to w_{base} and inversely proportional to its weight w_i (vruntime)
- w_{base} usually set at 100
- Task are scheduled in order of increasing vruntime

= Andrea Righi - NVIDIA -

$$vruntime(T_i) = \frac{w_{base}}{w_i} \cdot (t_1 - t_0) = \frac{100}{w_i} \cdot (t_1 - t_0)$$

EEVDF : Earliest Eligible Virtual Deadline First

- Lag : difference between the ideal runtime and actual runtime of task
- Eligibility : Lag >=0 for task eligibility to run
- Virtual Deadline : vruntime + requested runtime

- Andrea Righi - NVIDIA -

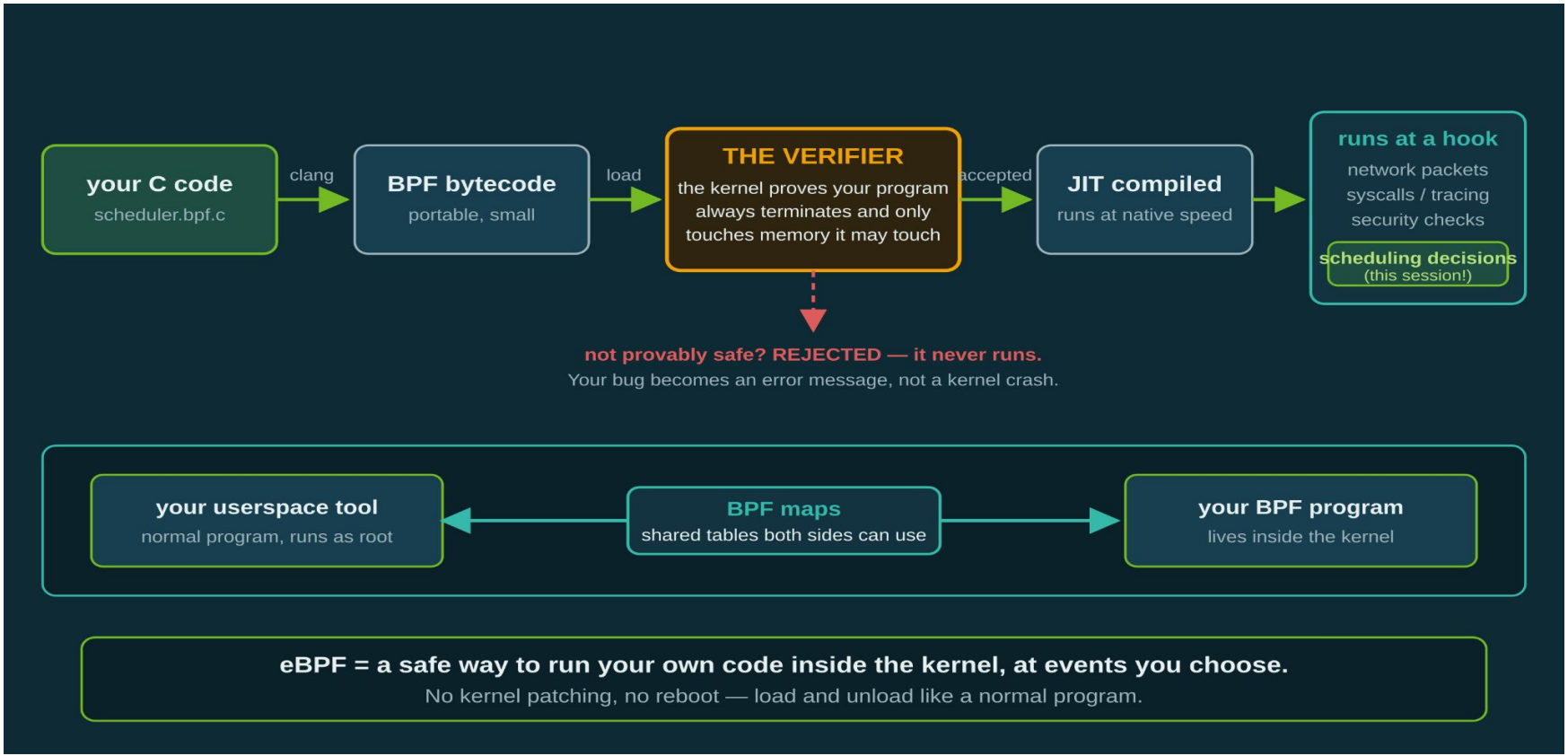
$$lag_T(t_1) = V_{avg}(t_1) - V_T(t_1) \geq 0$$

$$D_T(t_1) = V_T(t_1) + \Delta t_T \cdot \frac{w_{base}}{w_i}$$

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

eBPF 101

eBPF Life Cycle



eBPF 101 for This Session

BPF program

A small function you write in C. The kernel calls it when something happens — for us: scheduling events.

hook

The event your program is attached to. This session's hooks: "a task woke up", "a CPU wants work".

verifier

The kernel's proof-checker. It rejects any program that could loop forever or read the wrong memory — before it ever runs.

map

A shared table (array, hash, per-task storage) that both the BPF program and your userspace tool can read and write.

kfunc

A kernel function BPF is allowed to call. Everything starting with scx_bpf_* in this session is a kfunc.

struct_ops

A struct full of BPF programs implementing a kernel interface. A sched_ext scheduler is exactly one struct_ops.

BPF Program Types

```
enum bpf_prog_type {
    BPF_PROG_TYPE_UNSPEC,
    BPF_PROG_TYPE_SOCKET_FILTER,
    BPF_PROG_TYPE_KPROBE,
    BPF_PROG_TYPE_SCHED_CLS,
    BPF_PROG_TYPE_SCHED_ACT,
    BPF_PROG_TYPE_TRACEPOINT,
    BPF_PROG_TYPE_XDP,
    BPF_PROG_TYPE_PERF_EVENT,
    BPF_PROG_TYPE_CGROUP_SKB,
    BPF_PROG_TYPE_CGROUP_SOCK,
    BPF_PROG_TYPE_LWT_IN,
    BPF_PROG_TYPE_LWT_OUT,
    BPF_PROG_TYPE_LWT_XMIT,
    BPF_PROG_TYPE_SOCK_OPS,
    BPF_PROG_TYPE_SK_SKB,
    BPF_PROG_TYPE_CGROUP_DEVICE,
    BPF_PROG_TYPE_SK_MSG,
    BPF_PROG_TYPE_RAW_TRACEPOINT,
    BPF_PROG_TYPE_CGROUP_SOCK_ADDR,
    BPF_PROG_TYPE_LWT_SEG6LOCAL,
    BPF_PROG_TYPE_LIRC_MODE2,
    BPF_PROG_TYPE_SK_REUSEPORT,
    BPF_PROG_TYPE_FLOW_DISSECTOR,
    /* See /usr/include/linux/bpf.h f
       the full list. */
};
```

Enabling eBPF

```
# BPF core
CONFIG_BPF=y
CONFIG_BPF_SYSCALL=y
CONFIG_BPF_JIT=y
CONFIG_BPF_JIT_ALWAYS_ON=y
CONFIG_BPF_JIT_DEFAULT_ON=y

# BTF type info (CO-RE + struct_ops)
# needs pahole >= 1.25 ("dwarves" pkg)
CONFIG_DEBUG_INFO=y
CONFIG_DEBUG_INFO_BTF=y
```


sched_ext

sched_ext implementation

- Scheduling Policy that runs below the fair sched class. Utilizing eBPF for rapid development and experimentation
- Initiated and maintained by a group of engineers from Meta
- When a BPF scheduler loaded, all fair sched class tasks are moved into sched_ext
- sched_ext invokes BPF callback via struct_ops

sched_ext implementation

- When BPF scheduler experiencing faulty, or exit - all tasks will be returned to fair sched
- Repo https://github.com/sched-ext/scx

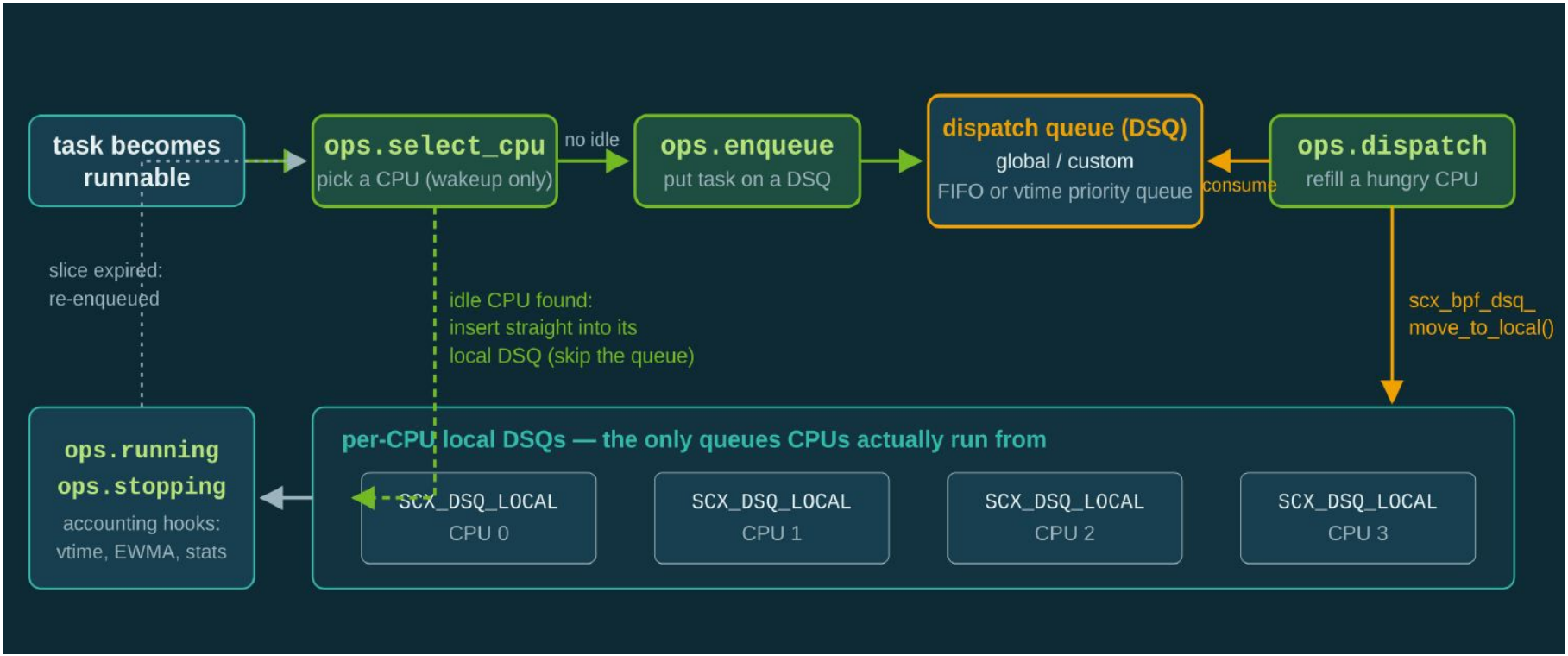
BPF Scheduler Implementation

```
SCX_OPS_DEFINE(vder_ops,
               .select_cpu          = (void *)vder_select_cpu,
               .enqueue             = (void *)vder_enqueue,
               .dispatch            = (void *)vder_dispatch,
               .runnable            = (void *)vder_runnable,
               .running             = (void *)vder_running,
               .stopping            = (void *)vder_stopping,
               .quiescent           = (void *)vder_quiescent,
               .enable              = (void *)vder_enable,
               .init_task           = (void *)vder_init_task,
               .cpu_release         = (void *)vder_cpu_release,
               .init                = (void *)vder_init,
               .exit                = (void *)vder_exit,
               .timeout_ms          = 5000,
               .name                = "vder");
```

DSQ - Dispatch Queue

- The basic building block of scheduler profiles - the data structures to implement queueing profiles algorithm
- Kind :
  - SCX_DSC_LOCAL : one on each cpu core
  - SCX_DSC_GLOBAL : global dispatch queue
  - User defined DSQ

DSQ - Dispatch Queue



The Three Scheduler

The Three Scheduler

	1 · scx_minfifo global FIFO	2 · scx_toydsq custom DSQ + vtime fairness	3 · scx_prefcore AMD preferred-core steering
select_cpu	— kernel default —	idle CPU? → direct dispatch	scan CPUs in ranking order
enqueue	insert → GLOBAL (FIFO)	vtime insert → SHARED_DSQ	classify → LATENCY / BATCH DSQ
dequeue	no-op (shown for teaching)	—	—
dispatch	no-op — GLOBAL auto-consumed	consume SHARED_DSQ	tiered consume + fallback
running/stopping	—	vtime clock / charge by weight	stopping: EWMA of used slice
tick	—	—	sample cpuperf_cur()
enable/init_task	—	enable: seed task vtime	init_task: create task ctx
init	—	create SHARED_DSQ	create both DSQs
exit	record exit info (UEI)	record exit info (UEI)	record exit info (UEI)

scx_minfifo : Just a Simple FIFO

```
void BPF_STRUCT_OPS(minfifo_enqueue,
        struct task_struct *p, u64 enq_flags)
{
    scx_bpf_dsq_insert(p, SCX_DSQ_GLOBAL,
                       SCX_SLICE_DFL, enq_flags);
}

void BPF_STRUCT_OPS(minfifo_dispatch, s32 cpu,
        struct task_struct *prev)
{
    /* empty: the kernel consumes GLOBAL
       before it even calls us */
}

SCX_OPS_DEFINE(minfifo_ops,
    .enqueue  = (void *)minfifo_enqueue,
    .dispatch = (void *)minfifo_dispatch,
    .exit     = (void *)minfifo_exit,
    .name     = "minfifo");
```

- No Fairness, No Locality. Just a FIFO
- Every fallback skips to the kernel default

scx_toydsq : Nice-ing Virtual Runtime

```
s32 BPF_STRUCT_OPS(toydsq_select_cpu, ...)
{
    cpu = scx_bpf_select_cpu_dfl(p, prev_cpu,
                    wake_flags, &is_idle);
    if (is_idle)  /* skip the queue! */
        scx_bpf_dsq_insert(p, SCX_DSQ_LOCAL, ...);
    return cpu;
}

void BPF_STRUCT_OPS(toydsq_dispatch, ...)
{   /* mandatory now */
    scx_bpf_dsq_move_to_local(SHARED_DSQ, 0);
}

s32 BPF_STRUCT_OPS_SLEEPABLE(toydsq_init)
{
    return scx_bpf_create_dsq(SHARED_DSQ, -1);
}
```

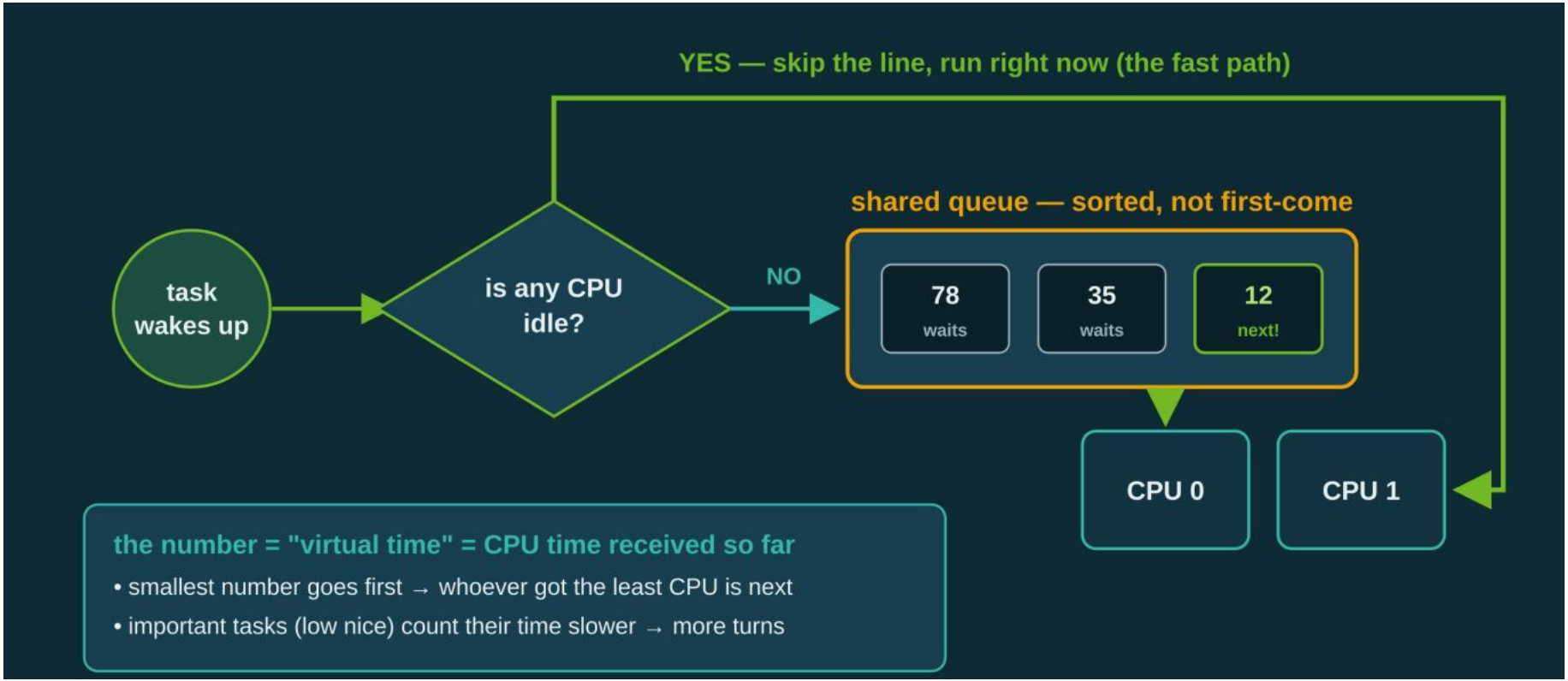
- Find Idle CPU
- Direct Dispatch to Idle CPU
- Global Dispatch Queue with Nice-ing Virtual Runtime
- Global Dispatch to Local Queue

scx_toydsq : Nice-ing Virtual Runtime

```
/*
 * ops.stopping - task p is coming off a CPU. Charge it for the slice it
 * actually used, scaled inversely by weight: at nice 0 (weight 100) real
 * time == vtime; a heavier task's vtime advances slower, so it gets
 * proportionally more CPU. This one line is the whole fairness policy.
 */
void BPF_STRUCT_OPS(toydsq_stopping, struct task_struct *p, bool runnable)
{
    p->scx.dsq_vtime +=
        (SCX_SLICE_DFL - p->scx.slice) * 100 / p->scx.weight;
}
```

- Find Idle CPU
- Direct Dispatch to Idle CPU
- Global Dispatch Queue with Nice-ing Virtual Runtime

scx_toydsq : Nice-ing Virtual Runtime



openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

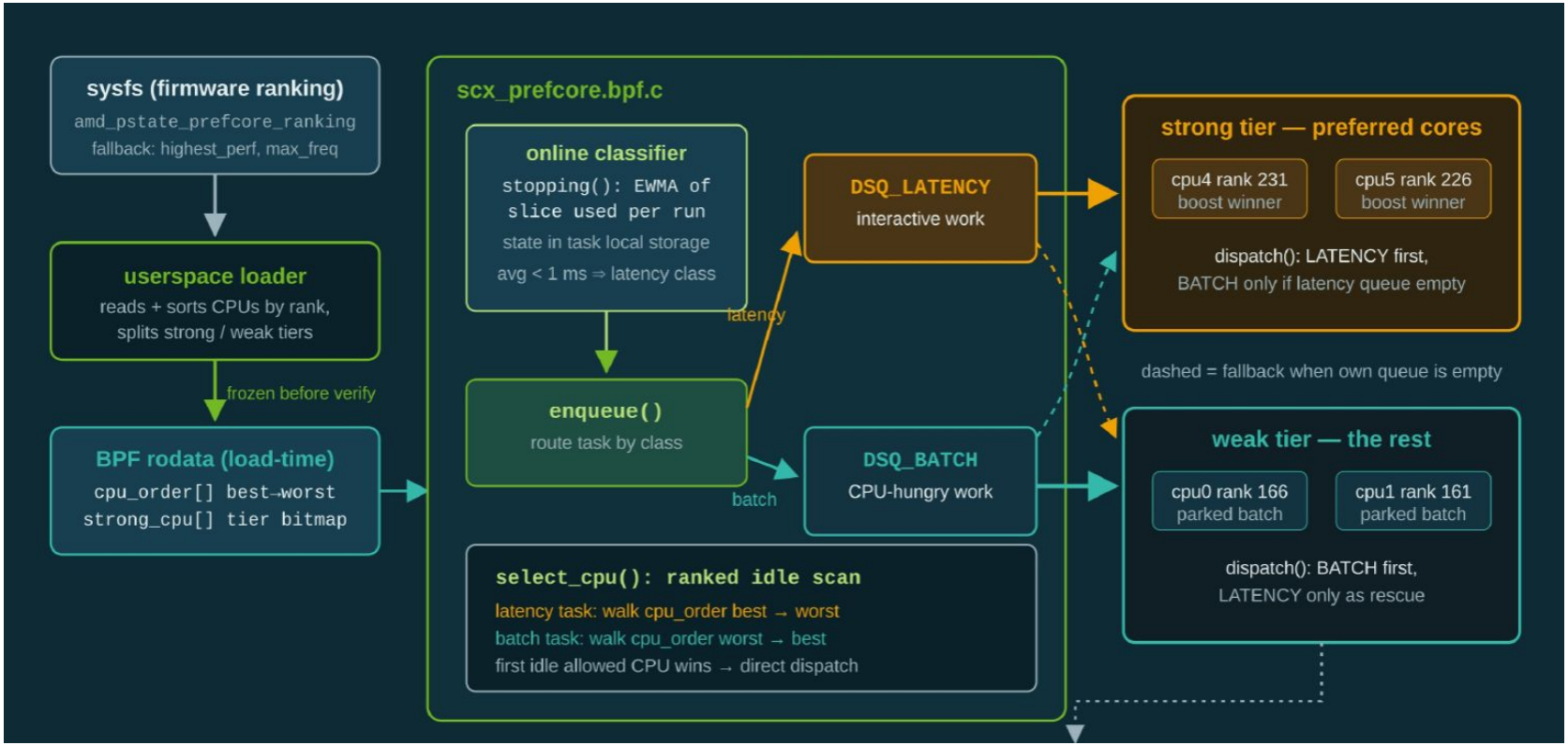
scx_prefcore : AMD Preferred Core Ranking

```
~ (0.007s)
grep . /sys/devices/system/cpu/cpu*/cpufreq/\
amd_pstate_prefcore_ranking

/sys/devices/system/cpu/cpu0/cpufreq/amd_pstate_prefcore_ranking:191
/sys/devices/system/cpu/cpu10/cpufreq/amd_pstate_prefcore_ranking:186
/sys/devices/system/cpu/cpu11/cpufreq/amd_pstate_prefcore_ranking:181
/sys/devices/system/cpu/cpu12/cpufreq/amd_pstate_prefcore_ranking:176
/sys/devices/system/cpu/cpu13/cpufreq/amd_pstate_prefcore_ranking:196
/sys/devices/system/cpu/cpu14/cpufreq/amd_pstate_prefcore_ranking:166
/sys/devices/system/cpu/cpu15/cpufreq/amd_pstate_prefcore_ranking:171
/sys/devices/system/cpu/cpu1/cpufreq/amd_pstate_prefcore_ranking:196
/sys/devices/system/cpu/cpu2/cpufreq/amd_pstate_prefcore_ranking:186
/sys/devices/system/cpu/cpu3/cpufreq/amd_pstate_prefcore_ranking:181
/sys/devices/system/cpu/cpu4/cpufreq/amd_pstate_prefcore_ranking:176
/sys/devices/system/cpu/cpu5/cpufreq/amd_pstate_prefcore_ranking:196
/sys/devices/system/cpu/cpu6/cpufreq/amd_pstate_prefcore_ranking:166
/sys/devices/system/cpu/cpu7/cpufreq/amd_pstate_prefcore_ranking:171
/sys/devices/system/cpu/cpu8/cpufreq/amd_pstate_prefcore_ranking:191
/sys/devices/system/cpu/cpu9/cpufreq/amd_pstate_prefcore_ranking:196
```

- Silicon Lottery : some cores sustain higher boost clocks and exposed by pstate into kernel
- Organize CPU by its boost ranking

scx_prefcore : AMD Preferred Core Ranking



Demo : Run The Scheduler

Codebase

- Go to : https://github.com/tonnyadhi/scx-talk-osas-2026

Drop In Compilation into scx_repo

```
$ git clone https://github.com/sched-ext/scx.git
$ cd scx
$ cp scx-talk-demos/scx_*.c        scheds/c/
$ cp scx-talk-demos/scx_*.bpf.c    scheds/c/

# register the schedulers: edit scheds/c/meson.build
c_scheds = [ 'scx_simple', ...,
  'scx_minfifo', 'scx_toydsq', 'scx_prefcore' ]

$ meson setup build --prefix ~
$ meson compile -C build

# binaries:
build/scheds/c/scx_minfifo
build/scheds/c/scx_toydsq
build/scheds/c/scx_prefcore
```

Run and Observe The Scheduler

```
$ sudo ./scx_prefcore -s 4      # root: loads BPF

# is it in charge right now?
$ cat /sys/kernel/sched_ext/state
enabled
$ cat /sys/kernel/sched_ext/root/ops
prefcore

# what is loaded, and how much does it cost?
$ sudo bpftool struct_ops list
$ sudo sysctl kernel.bpf_stats_enabled=1
$ sudo bpftool prog show | grep -A1 enqueue
  run_cnt 184223  run_time_ns 91ms total
```





openSUSE
Asia Summit 2026
Yogyakarta, October 3 - 4

Q&A



Thank You



openSUSE®
Asia Summit 2026